

01 — THIẾT KẾ TEST CASE (Phase 1)

- **Sinh viên:** Ninh Văn Khải — MSSV **23127060**
- **Nguồn expected:** đề bài HW04 kết hợp với **source thật** của SUT — mọi ô "Expected" em đều dẫn về số

dòng cụ thể trong file

- **Input:** `report/00-SUT-RECON.md`

Ở tài liệu này em thiết kế bảng test case cho cả ba feature mà em phụ trách. Có một nguyên tắc em tự đặt ra và giữ xuyên suốt: **em không tự nghĩ ra kết quả mong đợi trong đầu**, mà mọi giá trị ở cột "Expected" em đều phải đọc được từ một dòng cụ thể trong source của SUT và ghi kèm số dòng đó lại. Em làm vậy để sau này

khi bảo vệ, em giải thích được vì sao em kỳ vọng như thế, và nếu em sai thì thầy/cô cũng truy ngược lại được.

Chú thích loại test case: **P** là positive, **N** là negative, **E** là edge/boundary, và **S** là security.

Các assertion pattern em sử dụng (theo §4.3 của SKILL):

Mã	Pattern	Ví dụ
A1	UI state / text	<code>expect(page.getByRole('heading', { name: '...' })).toBeVisible()</code>
A2	Navigation / URL	<code>expect(page).toHaveURL(/\/login/)</code>
A3	API / back-end state	<code>expect(res.status()).toBe(200)</code> ; verify qua <code>GET /api/orders/:id</code>
A4	Số học / boundary	<code>expect(order.total_amount).toBe(expectedTotal)</code>
A5	Dialog (alert)	<code>page.on('dialog', d => ...)</code> — SUT dùng <code>alert()</code> thay toast

1. FR-03 — Quên mật khẩu & Đặt lại mật khẩu (16 TC)

Với feature này, em thiết kế 16 test case. Em tập trung khá nhiều vào nhóm security (9 case), vì khi đọc source em thấy luồng đặt lại mật khẩu của SUT có nhiều chỗ hở nghiêm trọng.

Em rút ra các acceptance criteria sau từ mô tả FR-03 kết hợp với source:

1. Khi người dùng nhập một email đã đăng ký, hệ thống phải phát ra mã đặt lại mật khẩu cho email đó.
2. Khi người dùng nhập đúng mã kèm một mật khẩu mới hợp lệ, hệ thống phải đổi mật khẩu thành công, và sau

đó người dùng phải đăng nhập được bằng mật khẩu mới.

1. Nếu mã hoặc email không đúng, hệ thống phải từ chối yêu cầu và giữ nguyên mật khẩu cũ.
2. Mã đặt lại phải được giữ bí mật, chỉ dùng được một lần và phải có thời hạn sử dụng.

ID	Loại	Tiền đề	Bước	Dữ liệu	Expected (nguồn)	Assert	Auto?
FR03-TC01	P	User mới đã đăng ký qua API	/forgot-password → nhập email → Lấy mã OTP → đọc OTP từ hộp xanh → nhập OTP + mật khẩu mới → Đặt lại mật khẩu	pw = New Pass 123	alert Đổi mật khẩu thành công! , điều hướng /login , login API bằng pw mới ⇒ 200 (ForgotPassword.jsx:34-35)	A5+A2+A3	✓
FR03-TC02	N	—	Nhập email chưa đăng ký → Lấy mã OTP	nobody-...@nope.test	alert chứa User not found , vẫn ở bước 1 (server.js:69 , ForgotPassword.jsx:22)	A5+A1	✓
FR03-TC03	N	User mới, đã lấy OTP	Nhập OTP sai (4 số khác) + pw mới	otp sai	alert Mã OTP không đúng hoặc có lỗi xảy ra. ; login bằng pw cũ vẫn 200 (server.js:92)	A5+A3	✓
FR03-TC04	N	User mới, đã lấy OTP	Nhập OTP đúng + NewPass123! (có ký tự đặc biệt)	NewPass123!	🚫 alert Mật khẩu quá yếu!... — FE chặn mật khẩu MẠNH vì regex đòi khoảng trắng và cấm ký tự đặc biệt (ForgotPassword.jsx:26) ⇒ BUG-03-01	A5+A3	✓
FR03-TC05	E	User mới, đã lấy OTP	Boundary độ dài regex FE: Ab 12345 (8 ký tự, hợp lệ) và Ab 1234 (7 ký tự)	2 biến thể	8 ký tự ⇒ thành công; 7 ký tự ⇒ alert quá yếu (ForgotPassword.jsx:26 {8,})	A5+A4	✓
FR03-TC06	S	User mới	POST /api/forgot-password	email hợp lệ	🚫 body chứa resetToken — mã bí mật lộ trong HTTP response (server.js:76-79) ⇒ BUG-03-02	A3	✓
FR03-TC07	S	User mới	Đọc resetToken trả về	—	🚫 token khớp /\d{4}\$/ ⇒ chỉ 9000 tổ hợp, brute-force được (server.js:70) ⇒ BUG-03-03	A3+A4	✓
FR03-TC08	S	User mới	Gọi POST /api/forgot-password 10 lần liên tiếp	10 request	🚫 cả 10 đều 200 , không rate-limit, không CAPTCHA (server.js:66-82 không có middleware) ⇒ BUG-03-04	A3	✓
FR03-TC09	S	User mới	Gọi forgot-password với email tồn tại và email không tồn tại	2 email	🚫 200 vs 404 ⇒ phân biệt được tài khoản nào có thật (user enumeration) (server.js:69) ⇒ BUG-03-05	A3	✓

ID	Loại	Tiền đề	Bước	Dữ liệu	Expected (nguồn)	Assert	Auto?
FR03-TC10	S	User mới, có OTP	POST <code>/api/reset-password</code> với <code>newPassword = "1"</code>	1	 200 Password reset successfully + login bằng 1 thành công — backend không có policy mật khẩu (<code>server.js:84-95</code>) ⇒ BUG-03-06	A3	✓
FR03-TC11	S	Đã reset xong, có token đăng nhập	GET <code>/api/users/me</code>	—	 field password trả về đúng plaintext vừa đặt — không hash (<code>database.js:88</code> , <code>server.js:23</code>) ⇒ BUG-03-07	A3	✓
FR03-TC12	E	User mới	Gọi forgot-password 2 lần , reset bằng token lần 1	tokenA, tokenB	400 Invalid token or email — token cũ bị ghi đè (<code>server.js:72-75</code>). Hành vi ĐÚNG , test bảo vệ chống hồi quy	A3	✓
FR03-TC13	S	Đã reset thành công 1 lần	Dùng lại chính token đó để reset lần 2	token đã dùng	400 — reset_token đã bị set NULL (<code>server.js:87</code>). Hành vi ĐÚNG	A3	✓
FR03-TC14	S	User bị khoá (2 lần login sai ⇒ <code>login_attempts = 4 ≥ 3</code>)	Reset mật khẩu thành công → login bằng mật khẩu mới	—	 vẫn 403 Tài khoản đã bị khóa — reset không xóa <code>locked_until</code> (<code>server.js:86</code> chỉ update <code>password</code> , <code>reset_token</code>) ⇒ BUG-03-08	A3	✓
FR03-TC15	N	User mới	POST <code>/api/reset-password</code> với <code>resetToken</code> rỗng / không phải số	"", "abcd"	400 Invalid token or email (<code>server.js:91-92</code>)	A3	✓
FR03-TC16	S	User A và user B đều tồn tại	Dùng token của A để reset mật khẩu của B	tokenA + emailB	400 — điều kiện <code>WHERE email = ? AND reset_token = ?</code> chặn (<code>server.js:86</code>). Hành vi ĐÚNG	A3	✓

Những case em không automate được, và lý do cụ thể:

Em xin liệt kê thẳng ra đây những case mà em cân nhắc nhưng cuối cùng không automate được. Với mỗi case em

đều ghi rõ lý do kỹ thuật, để thầy/cô thấy là em không né tránh mà thực sự SUT không cho em cách nào kiểm tra:

Case	Lý do
Kiểm tra email đặt lại thật sự gửi tới hộp thư	SUT không gửi email , trả token thẳng trong response — không có kênh mail để verify
Token hết hạn sau N phút	<code>server.js</code> không lưu thời điểm tạo token ⇒ không có expiry để chờ; chờ thật sẽ làm test chạy hàng chục phút
CAPTCHA / chống bot	SUT không có
Mở khoá tài khoản sau 3 phút (<code>locked_until</code>)	Phải chờ 180 giây thật ⇒ vi phạm quy tắc cấm <code>waitForTimeout</code> dài; chỉ assert trạng thái đang khoá (TC14)

2. FR-08 — Thanh toán (Checkout) (18 TC)

Với feature thanh toán, em thiết kế 18 test case. Đây là feature em thấy rủi ro nhất về mặt nghiệp vụ, vì mọi lỗi ở đây đều quy ra tiền, nên em dành nhiều case cho nhóm security và boundary.

Em rút ra các acceptance criteria sau:

1. Chỉ những người đã đăng nhập mới được phép thanh toán.
2. Nếu giỏ hàng đang trống thì người dùng không thể tiến hành thanh toán.
3. Tổng tiền của đơn hàng phải do **server** tự tính ra, và phải khớp với giá sản phẩm nhân số lượng.
4. Một mã giảm giá hợp lệ phải làm **giảm** tổng tiền theo đúng công thức, đồng thời phải tôn trọng các

điều kiện về giá trị đơn tối thiểu, hạn sử dụng và số lượt được dùng.

1. Mỗi người dùng chỉ được xem đơn hàng của chính mình, không xem được đơn của người khác.

ID	Loại	Tiền đề	Bước	Dữ liệu	Expected (nguồn)	Assert	Auto?
FR08-TC01	P	User mới đã login qua UI	Home → Thêm vào giỏ → /cart → Tiến hành thanh toán → Xác Nhận Thanh Toán	SP seed id 1	heading Thanh toán thành công! ; GET /api/orders/my-orders có đơn mới, total_amount = giá SP (Checkout.jsx:70)	A1+A3+A4	✓
FR08-TC02	N	Chưa thêm gì	Mở /cart	—	text Giỏ hàng của bạn đang trống (Cart.jsx:24)	A1	✓
FR08-TC03	N	Chưa đăng nhập , giỏ có hàng	/cart → Tiến hành thanh toán	—	alert Bạn cần đăng nhập để thanh toán! + URL /login (Cart.jsx:11-15)	A5+A2	✓
FR08-TC04	S	Login, giỏ có SP 30.000.000 đ	Ở /checkout sửa ô số về 1 → Xác Nhận Thanh Toán	1	🐛 thành công, đơn lưu total_amount = 1 — ô tổng tiền sửa được và backend tin client (Checkout.jsx:96-106 , server.js:302-317) ⇒ BUG-08-01	A1+A3+A4	✓
FR08-TC05	S	User đã login (API)	POST /api/checkout total_amount = -1000000	số âm	🐛 200 + đơn có tổng âm — không validate (server.js:305) ⇒ BUG-08-02	A3+A4	✓
FR08-TC06	N	User đã login (API)	POST /api/checkout total_amount = "abc"	chuỗi	🐛 200 , DB lưu giá trị vô nghĩa — cột INTEGER nhưng SQLite không ép kiểu (database.js:71) ⇒ BUG-08-02	A3	✓
FR08-TC07	S	User đã login, chưa hề thêm gì vào giỏ	POST /api/checkout total_amount = 0	giỏ rỗng	🐛 200 tạo đơn — backend không kiểm tra giỏ (server.js:302-317 bỏ qua items) ⇒ BUG-08-03	A3	✓
FR08-TC08	S	User A đã tạo đơn	User B (và cả request không token) gọi GET /api/orders/<id của A>	orderId của A	🐛 200 trả đủ dữ liệu đơn của A — endpoint không có authenticateToken (server.js:340) ⇒ BUG-08-04 (IDOR)	A3	✓
FR08-TC09	E	Vừa thanh toán thành công qua UI	Quay lại /cart	—	🐛 giỏ vẫn còn hàng ⇒ thanh toán lặp — clearCart được import nhưng không gọi (Checkout.jsx:8) ⇒ BUG-08-05	A1+A3	✓
FR08-TC10	E	Thanh toán thành công qua UI	Đọc đơn vừa tạo qua API	—	🐛 shipping_address = null — FE không gửi trường này (Checkout.jsx:44-48) ⇒ BUG-08-06	A3	✓
FR08-TC11	S	Giỏ 30.000.000	Nhập SAVE10 → Áp dụng	SAVE10 (percent 10)	🐛 final_amount = 300.000.000 — công thức	A1+A3+A4	✓

ID	Loại	Tiền đề	Bước	Dữ liệu	Expected (nguồn)	Assert	Auto?
		đ. ở /checkout			dùng <code>total(1 - discount_value)</code> thay vì <code>total discount_value/100</code> , giảm giá thành tăng gấp 10 (<code>server.js:432-434</code>) ⇒ BUG-08-07		
FR08-TC12	P	Tổng 30.000.000 đ	Áp BIGBUY (fixed 50.000)	BIGBUY	<code>final_amount = 29.950.000</code> — nhánh <code>fixed</code> tính đúng (<code>server.js:435-436</code>)	A3+A4	✓
FR08-TC13	N	Tổng đủ điều kiện	Áp EXPIRED	EXPIRED	400 Mã giảm giá đã hết hạn (<code>server.js:427-429</code>)	A3+A1	✓
FR08-TC14	N	—	Áp mã không tồn tại	KHONGCOMA	404 Mã giảm giá không tồn tại hoặc đã bị vô hiệu hóa (<code>server.js:414-418</code>)	A3+A1	✓
FR08-TC15	E	—	Boundary <code>min_order_amount</code> của BIGBUY = 500.000: thử <code>499.999 / 500.000 / 500.001</code>	3 mốc	🚫 500.000 (đúng bằng mức tối thiểu) bị từ chối 400 vì code dùng <code>></code> thay vì <code>>=</code> (<code>server.js:421</code>) ⇒ BUG-08-08	A3+A4	✓
FR08-TC16	E	User mới, đã dùng VIP100 2 lần	Áp VIP100 lần 3	VIP100 max 2	400 Bạn đã sử dụng mã này 2 lần... (<code>server.js:452-456</code>)	A3	✓
FR08-TC17	E	Giỏ có hàng	<code>reload()</code> trang <code>/cart</code>	—	🚫 giỏ rỗng sau reload — <code>CartContext</code> giữ state in-memory, không localStorage (<code>CartContext.jsx:7</code>) ⇒ BUG-08-09	A1	✓
FR08-TC18	S	—	POST <code>/api/checkout</code> không gửi <code>Authorization</code>	—	401 Unauthorized (<code>server.js:97-101</code>). Hành vi ĐÚNG — test chống hồi quy	A3	✓

Những case em không automate được, và lý do cụ thể:

Case	Lý do
Thanh toán qua cổng thanh toán thật (VNPay/Momo)	SUT không tích hợp cổng nào, <code>/api/checkout</code> chỉ ghi DB
Trừ tồn kho khi đặt hàng	Bảng <code>products</code> không có cột stock (<code>database.js:62-69</code>) ⇒ không có gì để assert
Gửi email xác nhận đơn hàng	SUT không gửi email
Race condition 2 người mua cùng lúc món cuối	Không có tồn kho ⇒ không có điều kiện tranh chấp

3. FR-15 — Quản lý sản phẩm (Admin CRUD) (20 TC)

Với feature quản lý sản phẩm, em thiết kế 20 test case. Ngoài các case CRUD thông thường, em chú ý nhiều tới phần kiểm soát truy cập và phần validate dữ liệu đầu vào, vì khi đọc source em thấy backend gần như không kiểm tra gì cả.

Em rút ra các acceptance criteria sau:

1. Chỉ tài khoản có quyền **admin** mới được phép thêm, sửa hoặc xóa sản phẩm.
2. Khi thêm một sản phẩm hợp lệ, sản phẩm đó phải xuất hiện trong danh sách và phải được lưu đúng vào

cơ sở dữ liệu.

1. Khi sửa một sản phẩm, **chi riêng** sản phẩm đó được thay đổi, các sản phẩm còn lại phải giữ nguyên.
2. Khi xóa một sản phẩm, sản phẩm đó phải biến mất khỏi danh sách.
3. Dữ liệu không hợp lệ, chẳng hạn tên rỗng, giá âm hoặc giá không phải là số, đều phải bị hệ thống từ chối.

ID	Loại	Tiền đề	Bước	Dữ liệu	Expected (nguồn)	Assert	Auto?
FR15-TC01	P	Admin app đang chạy	Login UI admin@eshop.com/Admin123! → sidebar Sản phẩm	seed admin	heading Quản lý Sản phẩm hiện, bảng có ≥5 dòng seed (App.jsx:339)	A1	✓
FR15-TC02	N	—	Login UI bằng test@eshop.com (role user)	seed user	alert Bạn không phải là admin! , vẫn ở màn login (App.jsx:64-67)	A5+A1	✓
FR15-TC03	P	Đã vào tab Sản phẩm	Điền form → Lưu sản phẩm	tên duy nhất, giá 1.234.000	dòng mới xuất hiện trong bảng; GET /api/products có SP đúng tên+giá (App.jsx:114-115)	A1+A3	✓
FR15-TC04	P	Đã có 1 SP do test tạo	Bấm Sửa ở dòng đó → đổi tên → Lưu sản phẩm	tên mới	alert Cập nhật thành công! ; API cho thấy SP đó đã đổi tên (App.jsx:113)	A5+A3	✓
FR15-TC05	S	Bảng có ≥3 SP, vừa sửa 1 SP	Đếm số dòng mang tên mới trong bảng	—	>1 dòng cùng tên — FE gán tên cho TẤT CẢ sản phẩm trong state (App.jsx:107-112); API chứng minh DB chỉ đổi 1 ⇒ BUG-15-01 (fake mass update)	A1+A3+A4	✓
FR15-TC06	P	Có SP do test tạo	Bấm Xóa ở dòng đó	—	dòng biến mất khỏi bảng; GET /api/products không còn id đó (App.jsx:127-134)	A1+A3	✓
FR15-TC07	S	—	POST /api/products không header Authorization	SP hợp lệ	200 {"message":"Product created"} — route thiếu authenticateToken (server.js:161) ⇒ BUG-15-02	A3	✓
FR15-TC08	S	Có SP do test tạo	PUT /api/products/:id không token	tên mới	200 Product updated (server.js:173) ⇒ BUG-15-02	A3	✓
FR15-TC09	S	Có SP do test tạo	DELETE /api/products/:id không token	—	200 Product deleted + SP mất thật khỏi DB (server.js:186) ⇒ BUG-15-02	A3	✓
FR15-TC10	S	User thường đã login	POST /api/products với token role = user	SP hợp lệ	200 — không kiểm tra role ở bất kỳ route product nào ⇒ BUG-15-03 (leo thang đặc quyền)	A3	✓
FR15-TC11	N	—	POST /api/products với name = ""	tên rỗng	200 + SP tên rỗng nằm trong danh sách	A3	✓

ID	Loại	Tiền đề	Bước	Dữ liệu	Expected (nguồn)	Assert	Auto?
					(server.js:162-171 không validate) ⇒ BUG-15-04		
FR15-TC12	N	—	POST /api/products với price = -5000	giá âm	200, GET trả về -5000 ⇒ BUG-15-05	A3+A4	✓
FR15-TC13	N	—	POST /api/products với price = "abc"	chuỗi	200, cột INTEGER nhưng SQLite lưu nguyên chuỗi ⇒ dữ liệu bẩn ⇒ BUG-15-05	A3	✓
FR15-TC14	E	—	POST /api/products với price = 9007199254740993 (> Number.MAX_SAFE_INTEGER)	boundary	200, giá trị đọc lại sai lệch do mất chính xác số nguyên ⇒ BUG-15-06	A3+A4	✓
FR15-TC15	N	—	DELETE /api/products/99999999	id không tồn tại	200 Product deleted thay vì 404 — không kiểm tra this.changes (server.js:186-191) ⇒ BUG-15-07	A3	✓
FR15-TC16	N	—	GET /api/products/99999999	id không tồn tại	200 {} thay vì 404 (server.js:157) ⇒ BUG-15-08	A3	✓
FR15-TC17	E	SP seed id 1.5	GET /api/products/2 và /api/products/1	id chẵn / lẻ	id chẵn trả price kiểu string , id lẻ trả number — bất nhất kiểu dữ liệu (server.js:158) ⇒ BUG-15-09	A3+A4	✓
FR15-TC18	N	—	POST /api/products với category_id = 999999	danh mục không tồn tại	200 — bảng products không có FOREIGN KEY (database.js:62-69) ⇒ BUG-15-10	A3	✓
FR15-TC19	S	—	Tạo SP tên rồi mở bảng admin	payload XSS	Payload lưu nguyên vẹn vào DB; bảng admin render bằng {p.name} nên React escape ⇒ không thực thi . Test khẳng định không có script chạy (dữ liệu độc vẫn tồn tại ⇒ rủi ro cho màn hình dùng dangerouslySetInnerHTML như App.jsx:801)	A1+A3	✓
FR15-TC20	E	Đã vào tab Sản phẩm	Bấm Lưu sản phẩm khi để trống Tên sản phẩm	—	Không có request nào được gửi — HTML5 required chặn ở client (App.jsx:496). Đây là lớp phòng thủ duy nhất , backend hoàn toàn không có (đối chiếu TC11)	A1+A3	✓

Những case em không automate được, và lý do cụ thể:

Case	Lý do
Upload ảnh sản phẩm thật	Form chỉ có ô nhập URL ảnh , không có upload file
Phân trang / sắp xếp danh sách	<code>App.jsx</code> render toàn bộ mảng, không có phân trang
Kiểm tra ảnh hiển thị đúng từ CDN	Phụ thuộc mạng ngoài (<code>placeholder.co</code>) ⇒ test sẽ flaky
Xoá SP đang nằm trong đơn hàng đã đặt	Bảng <code>orders</code> không lưu order_items (<code>database.js:71-78</code>) ⇒ không có quan hệ để kiểm tra dữ liệu mờ côi

4. Tổng hợp

Em xin tổng hợp lại số lượng test case đã thiết kế, phân theo từng loại:

Feature	Số TC	P	N	E	S	Bug dự kiến phát hiện
FR-03	16	1	4	2	9	8
FR-08	18	2	4	5	7	9
FR-15	20	4	7	4	5	10
Tổng	54	7	15	11	21	27

Rubric yêu cầu tối thiểu 12 test case cho mỗi feature, tức là 36 case cho cả ba. Em thiết kế được **54 case** nên đã vượt mức yêu cầu. Em cũng xin lưu ý là số lượng case ở bảng thiết kế này khác với số test thực thi trong code, vì khi triển khai em tách một số case ra thành nhiều biến thể data-driven.

5. Xác nhận duyệt của người học

- [x] Đã đọc và **duyet** 3 bảng test case trên (đề bài quy trách nhiệm review cho người học).
- [x] Đã **xác nhận** danh sách "không automate được" là hợp lý: mỗi dòng đều dẫn nguồn về một hạn chế

cụ thể của SUT (không gửi email, không lưu thời điểm phát token, `products` không có cột stock, `orders` không lưu order_items, form chỉ nhập URL ảnh) chứ không phải né việc.

- [x] **Ký tên:** Ninh Văn Khải — 23127060 · **Ngày:** 2026-08-27

Sau khi tự duyệt xong bảng này, em chuyển sang ****Phase 2** để sinh các file dữ liệu JSON và CSV ánh xạ một-một với bảng test case ở trên**. Phần đó em đã hoàn thành, tổng cộng 88 record nằm trong 6 file.